AdsKG-Agent: Knowledge-Graph RAG for Table Selection in a Retail-Media Warehouse

Capstone design doc v3 · AI Engineering Cohort · Anshika Chaurasia (solo) · October 2026

1. Problem Statement

Ad analysts ask questions such as *"CTR by brand for female users in age level 3 yesterday"* or *"unique reach of campaign 83237 in the last 6 hours"*. A retail-media warehouse holds many overlapping tables — the same metric at hourly/daily/weekly grain, raw vs. invalid-traffic-filtered, impressions vs. unique reach, real-time partial vs. reconciled. Picking the wrong variant returns a plausible but wrong number, so errors go unnoticed. **Hypothesis:** the dominant failure is *variant disambiguation and missing bridge tables*, not only context length. Baseline A tests this at the native size; a schema-scaling stress test tests context length.

Task: given one natural-language question (plus prior turns) and a fixed analysis clock `NOW = 2017-05-13 23:59 CST` (the last hour in the data), select the minimal correct tables and join path, generate DuckDB SQL, execute it, and answer with caveats. **In scope:** read-only questions over ~28 ad-performance tables; multi-turn follow-ups. **Out of scope:** root-cause ("why"), recommendations, writes, dashboards.

Success targets (fixed before building): chosen approach ≥ 75% test execution accuracy; ≥ +10 pp over vanilla vector RAG (B) on multi-hop + variant tiers; memory-on follow-up accuracy ≥ 80%; p50 latency ≤ 10 s; ≤ $0.02 per question.

Motivation: my company's ads analytics agent answers "what" questions; this prototypes the vector-DB, GraphRAG and memory layer it lacks, built fresh on public data (no company code or data).

2. Data & Ingestion Pipeline

Source (public)	Contents (verified on load)
Taobao Ad Display/Click (Alibaba, via Kaggle)	26.56M impressions, 1.37M clicks (CTR 5.14%), 1.14M users, May 6–13 2017; 847k ads → 423k campaigns, 256k advertisers, 99.8k brands, 6.8k categories; 1.06M user profiles (gender, age, spend level, city tier)
Stress-test distractors	TPC-DS (24 tables, generated in DuckDB) and BIRD dev schemas: added to the *catalog only* to grow it from ~28 to ~100 and ~250 tables; never part of a gold answer

- **Load & clean** (DuckDB): rename reserved-word column (`user`), parse literal "NULL" brands (29% of ads), flag 1.5M impressions with no user profile (LEFT JOIN required).
- **Model ~28 tables** with documented SQL: staging, 7 dimensions, event facts and variant families. Definitions are fixed before any benchmark question is written: *valid traffic* excludes users above a click-per-hour threshold chosen from the observed distribution; *real-time* tables hold the 6 h before NOW, while *reconciled* daily tables end the previous day (simulated 1-day lag, disclosed). Click-value = clicks × item price, labelled a proxy (no spend/revenue in the data). IDs are shown as-is ("brand 95471"); no invented names.
- **Catalog extraction:** table cards (grain, freshness, columns, 3 sample values, join keys) generated from information_schema + column stats; descriptions LLM-drafted, hand-reviewed and frozen before evaluation → Qdrant. Tables, columns, metrics, business terms and edges → Neo4j via idempotent MERGE.
- **Pivot already made:** the purchase `behavior_log` (needed for attribution variants) is absent from the Kaggle mirror and the Tianchi download failed; replaced by traffic-quality and reach variants.

3. Architecture

All data access goes through one **MCP server** ("adskg"), so B–E share identical tool code and differ only in orchestration. B–D are **fixed pipelines** (rewrite → retrieve → rerank → expand → SQL). E is a **LangGraph ReAct agent** (MCP client) that chooses the order, may re-query the catalog and repairs failed SQL, inside an explicit **agent harness**. Figure 1 shows E's typical path.

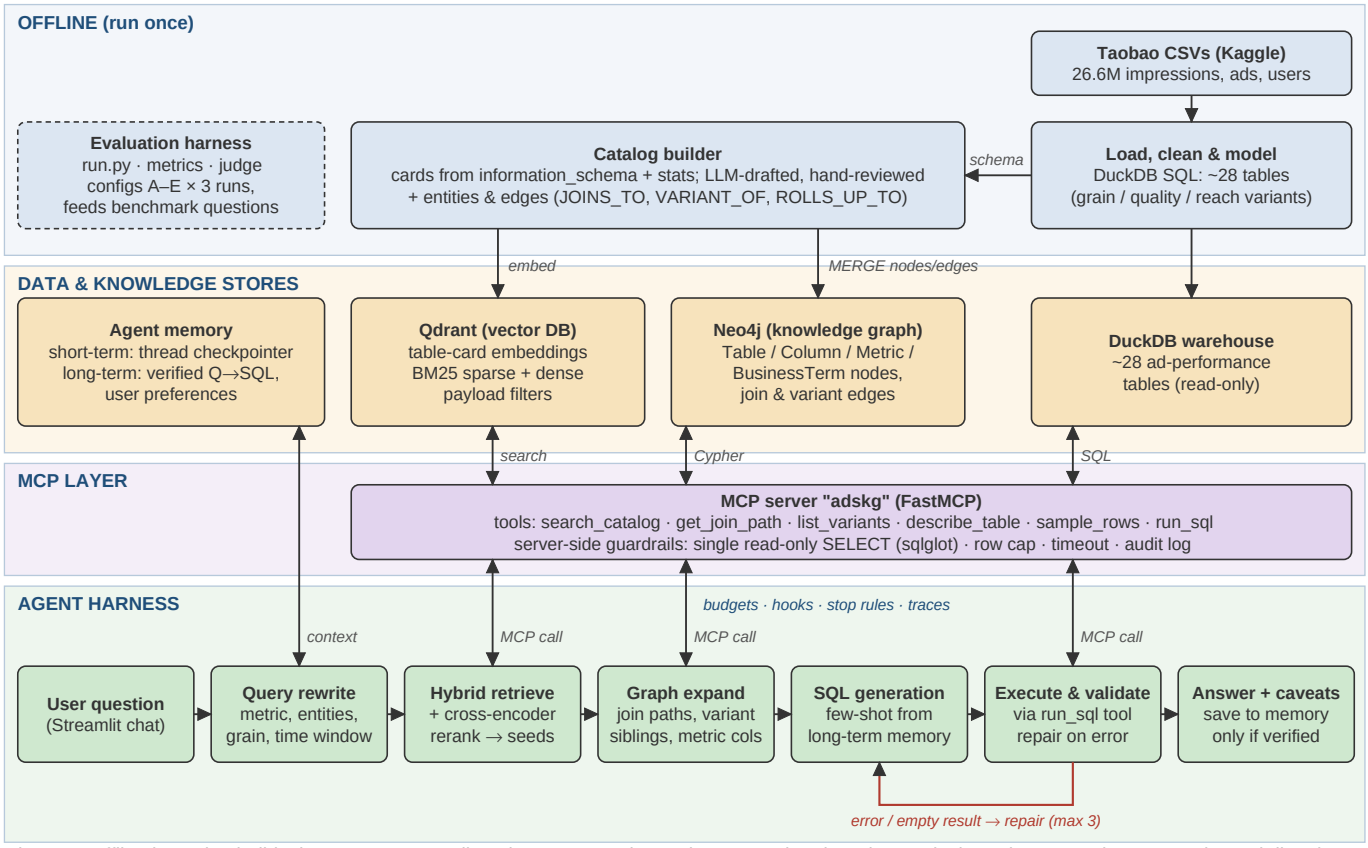

Figure 1. Offline ingestion builds the stores once; online, the agent reaches Qdrant, Neo4j and DuckDB only through MCP tools; memory is read directly from the LangGraph store. The evaluation harness replays the benchmark through this flow.

Component	Design
Knowledge graph	(:BusinessTerm)-[:MEANS]->(:Metric)-[:COMPUTED_FROM]->(:Column); (:Table)-[:HAS_COLUMN]->(:Column)-[:JOINS_TO]->(:Column); (:Table)-[:VARIANT_OF {differs_by}]->(:Table); [:ROLLS_UP_TO]; advertiser→campaign→adgroup hierarchy; (:VerifiedQuery)-[:USED]->(:Table)
Graph expansion	Cypher shortestPath between each pair of seed tables (≤ 3 hops), plus VARIANT_OF siblings with their differs_by attributes so the LLM chooses the variant; capped at 8 tables to limit over-expansion
MCP layer	Python FastMCP server (stdio locally, HTTP for the demo); tools search_catalog, get_join_path, list_variants, describe_table, sample_rows, run_sql; resource catalog://tables/{name}. Consumed by LangGraph via langchain-mcp-adapters and, unchanged, by Claude Desktop / Cursor in the demo
Agent harness	Budgets: ≤ 8 steps, ≤ 3 SQL repairs, token and $ cap per question. Hooks: pre-tool SQL validation, post-tool truncation to 50 rows, trace logging. Context: ≤ 8 reranked cards (~2k tokens), conversation summarised after 6 turns. Stop: verified result, or budget exhausted → "not sure" instead of a guess
Memory	Short-term: LangGraph checkpointer per thread ("now split that by age"). Long-term: verified Q→SQL pairs in Qdrant as few-shot examples, plus user preferences. **In evaluation, memory is seeded from the dev split only** (no test leakage)
Safety (in MCP server)	DuckDB read-only connection; sqlglot allows a single SELECT; LIMIT 1000; 20 s timeout; every call audit-logged. Enforced server-side, so no client can bypass it
Evaluation harness	Config-driven runner (approach, model, k, hops, memory on/off) → JSONL results → metrics, LLM judge and report scripts; every run traced in Langfuse for failure analysis

4. Approaches Compared (simple baseline first, one technique added per step)

ID	Approach	Isolates the value of
A	Full schema in prompt, one-shot SQL	Baseline
B	Vanilla vector RAG: dense top-k table cards	Vector DB retrieval
C	Advanced RAG: B + hybrid BM25 + rerank + query rewrite (each ablated separately if time permits)	Advanced retrieval
D	GraphRAG: C + Neo4j join-path and variant expansion	Knowledge graph
E	LangGraph agent using D's capabilities as MCP tools; run with and without memory	Agentic loop; memory (ablation)
E′	*Stretch:* same agent with off-the-shelf MCP servers (mcp-neo4j-cypher, mcp-server-qdrant, DuckDB MCP) instead of domain tools	Domain vs. generic tools

Final choice: highest test execution accuracy; if two approaches' 95% CIs overlap, the cheaper/faster one wins. Memory is judged by E-with vs. E-without memory, not against A–D.

5. Evaluation Criteria

- **Benchmark:** 120 questions (40 per tier: single-table, multi-hop join, variant/ambiguity) written from analyst-intent templates and **frozen before B–E are built**; where possible 20 written blind by a cohort peer to reduce author bias. Each has gold tables and gold SQL, executed and hand-checked. Split 40 dev / 80 test, plus 20 multi-turn conversations (5 dev / 15 test). Released in the repo.
- **Metrics:** *execution accuracy* = result sets match (order-insensitive unless ORDER BY, numeric tolerance 1e-6, column names ignored); *table set-F1* on tables used by the final SQL (all approaches) and *recall@k* of retrieval (B–E); join-path correctness; repair rate; p50/p95 latency, tokens, $ per question; follow-up accuracy.
- **Stress test:** execution accuracy and recall for A–D as the catalog grows 28 → ~100 tables (→ ~250 if time permits), showing where the full-schema baseline breaks.
- **Statistics & reproducibility:** 3 runs per configuration; 95% bootstrap CIs; paired McNemar test for each step A→B→C→D→E; every reported number is regenerated by the evaluation harness from saved traces.
- **LLM-as-judge:** a larger model different from the generator, blind to the approach; 1–5 rubric for relevance, caveat correctness and faithfulness to the SQL result; calibrated on 30 human-graded answers (report Cohen's κ).
- **Qualitative:** ≥ 30 failures hand-labelled into a taxonomy (wrong variant, missing bridge table, wrong join, over-expansion, hallucinated column, wrong time window) for the README failure analysis.

6. Framework Justification

Choice	Why	Alternative considered
DuckDB	Columnar and in-process; 26M rows aggregate in seconds on a laptop; no server	Postgres (needs a server)
Qdrant	Native hybrid sparse + dense search with payload filters (grain, freshness); embedded or Docker	Chroma, pgvector (simpler; hybrid support less mature)
Neo4j	Cypher path queries and persistence. At 28 tables NetworkX would suffice; Neo4j is chosen for the 100–250-table stress test and because it transfers to production work	NetworkX (in-memory, no query language)
LangGraph	Explicit state graph for the retrieve→SQL→validate→repair loop; built-in checkpointer and store for short/long-term memory	Pydantic AI, smolagents (less built-in memory/state)
Models	Small OpenAI chat model for generation; larger, different model as judge; text-embedding-3-small; open-source bge-reranker-base run locally	—
MCP (FastMCP)	One tool implementation shared by every approach and any MCP client (LangGraph via langchain-mcp-adapters); guardrails enforced server-side; mirrors how agents are integrated at work	Plain Python tool functions
Langfuse, sqlglot, Streamlit	Open-source tracing of every tool call for failure analysis; SQL parsing for the safety check; optional chat UI	LangSmith; regex; Gradio

7. Risks & Mitigations

- **Baseline A wins at 28 tables** → reported honestly as a finding; the stress test shows where retrieval becomes necessary.
- **Self-authored benchmark bias** → definitions and questions frozen before building; peer-written subset; tiers reported separately; benchmark released. **LLM-drafted descriptions leak hints** → hand-reviewed, frozen, identical for all approaches.
- **Many new tools in a short build** → must-have: A, B, D, E, MCP server, evaluation harness, 100-table stress test; should-have: C ablations, long-term memory; stretch: E′, 250 tables. **MCP overhead** → tool-call latency measured in the harness; stdio transport during evaluation.
- **Simulated freshness and value proxy** → disclosed; no question claims real spend or revenue.